

Reality AI — Person 2 (Backend Track) — Antigravity Prompt Playbook

Same format as Person 1's playbook: every prompt is a full spec — behavior, validation, error handling, logging, and test expectations spelled out — not a one-line request. Paste each into Antigravity in order; manual steps are interleaved. Re-read `AGENTS.md` before starting — every endpoint path, DB field name, and function signature below matches it exactly on purpose, since Person 1's AI modules and the frontend both depend on this contract without seeing your code.

Phase 0 — Foundation & Environment Setup (Week 1)

Manual 0.1 — Environment Install Python 3.11+, Git, Docker Desktop, and Antigravity. Create a new GitHub repo (becomes `backend/` at final merge). Clone it, open in Antigravity, copy `Reality-AI-AGENTS.md` in as `AGENTS.md`.

Prompt 0.1 — Folder skeleton

Read `AGENTS.md` fully first — it's the cross-team contract this backend must match exactly (API paths **and** response shapes, DB schema, environment variable names), since Person 1's AI modules and the frontend team both depend **on** it without seeing this repo's code.

Set up this folder skeleton, matching what becomes `backend/` at final merge:

```
backend/
├── app/
│   ├── core/
│   ├── models/
│   ├── schemas/
│   ├── api/
│   │   └── routes/
│   ├── services/
│   └── db/
│       └── migrations/
├── requirements.txt
└── .env.example
```

Add `__init__.py` to every Python package folder with a one-line module docstring. No implementation code yet.

Prompt 0.2 – Dependencies

Create `requirements.txt` with major-version pins and a one-line comment on each explaining its role: `fastapi`, `uvicorn[standard]`, `sqlalchemy`, `geoalchemy2`, `psycopg2-binary`, `pgvector`, `alembic`, `python-jose[cryptography]`, `passlib[bcrypt]`, `faker`, `python-dotenv`, `pydantic-settings`, `pytest`, `httpx`, `shapely` (for polygon validation later). Then create a virtual environment named `venv`, activate it, install everything, and report any failures clearly.

Prompt 0.3 – Config and database core

Build `app/core/config.py`: a `Settings` class (pydantic-settings `BaseSettings`) that loads `DATABASE_URL` and `SECRET_KEY` from `.env`, failing fast at startup with a clear error if either is missing – don't let the app run silently misconfigured. Add `JWT_ALGORITHM = "HS256"` and `JWT_EXPIRE_MINUTES = 60` as settings with defaults. Expose a module-level singleton `'settings = Settings()'`.

Build `app/core/database.py`: create a SQLAlchemy engine from `settings.DATABASE_URL`, a `SessionLocal` sessionmaker, and a declarative `Base`. Add `def get_db()`: a generator that yields a session and closes it in a `finally` block, for use as a FastAPI dependency.

Prompt 0.4 – Local Postgres via Docker

Create `docker-compose.yml`: a `postgis/postgis:16-3.4` service, with `POSTGRES_USER/PASSWORD/DB` read from environment variables (not hardcoded), a named volume for persistence, port `5432` exposed, and a healthcheck using `pg_isready`. Add `db/init.sql`, mounted into the `init` directory, that runs `CREATE EXTENSION IF NOT EXISTS postgis;` and `CREATE EXTENSION IF NOT EXISTS vector;` – actual tables come from Alembic, not this script.

Prompt 0.5 – Alembic setup

Initialize Alembic under `app/db/migrations`. Configure `alembic.ini` and `env.py` to read `DATABASE_URL` from `app.core.config.settings` rather than a hardcoded value, and to detect model changes automatically by importing `Base` from `app.core.database` and setting `target_metadata = Base.metadata`.

Don't generate any migrations yet – just verify `alembic current` runs cleanly against the local DB once it's up.

Prompt 0.6 – App entrypoint and env template

Build `app/main.py`: instantiate `FastAPI()` with a descriptive title/version, add `CORSMiddleware` configured permissively **for** local dev (comment noting this needs tightening before **any** real deployment), and a `GET / health-check` endpoint returning `{"status": "ok"}`. **Leave** commented-out placeholders **for** router includes – we'll add them **as** routes **get** built **in** later phases.

Create `.env.example` with `DATABASE_URL` and `SECRET_KEY` (exact names, per `AGENTS.md`), example placeholder values, and explanatory comments.

Create `.gitignore`: `.env`, `venv/`, `__pycache__`/, `*.pyc`, `.pytest_cache/`.

Manual 0.2 – Verify and commit `docker-compose up -d`, confirm the healthcheck passes, run `uvicorn app.main:app --reload`, open `http://localhost:8000/docs` and confirm it loads. Review everything Antigravity wrote, then `git add . && git commit -m "Initial backend/ skeleton" && git push`.

Phase 1 – Core Data & Listings Layer (Weeks 2-4)

Prompt 1.1 – Listing model

Build `app/models/listing.py` **using** the declarative Base **from** `app.core.database`:

```
class Listing(Base):
    __tablename__ = "listing"
    id: Integer, primary key, autoincrement
    broker_id: Integer, ForeignKey("broker.id"), not null, indexed
    title: String(200), not null
    description: Text, nullable
    price: Numeric(12, 2), not null
    property_type: String, not null – add a CheckConstraint restricting it to
        'flat' or 'house_land', matching AGENTS.md's enum exactly
    location: Geometry("POINT", srid=4326) from geoalchemy2, not null – every
        radius/boundary query filters on this
    carpet_area, built_up_area, plot_area: Numeric, nullable
    floor_number: Integer, nullable
```

```
rooms: JSON, nullable
embedding: Vector(<dimension Person 1 gave you>) via the pgvector
  SQLAlchemy integration, nullable – populated later by Person 1's
  pipeline, not by this repo
amenities: JSON, nullable – populated later by Person 1's amenities
  service, not by this repo
created_at: DateTime(timezone=True), server_default=func.now()
```

Add `__repr__` for debugging, and a comment above the class linking to AGENTS.md's schema section since this is a cross-team contract, not something to modify unilaterally.

Prompt 1.2 – Remaining models

Build `app/models/broker.py`, `app/models/customer.py`,
`app/models/conversation.py`, `app/models/lead.py`, one class per file:

Broker: `id`, `name` (`String`), `email` (`String`, unique, indexed, not null),
`phone_hash` (`String` – store hashed, never raw), `password_hash` (`String`, not
null), `created_at`.

Customer: `id`, `name`, `email` (unique, indexed), `created_at`. Add a comment
flagging that customers currently require an email – confirm this design
choice with the team rather than assuming it's final.

Conversation: `id`, `customer_id` (`ForeignKey`, indexed), `messages` (`JSON` – list
of {role, content, timestamp} objects), `created_at`, `updated_at`
(`server_default` and `onupdate` both `func.now()`).

Lead: `id`, `listing_id` (`ForeignKey`, indexed), `customer_id` (`ForeignKey`,
indexed), `created_at` – tracks a customer inquiry for broker analytics.

Add `relationship()` definitions (e.g. `Broker.listings`, `Listing.broker`) with
`back_populates` so related rows are easy to navigate in Python without
hand-written joins everywhere.

Manual 1.1 – First real migration `alembic revision --autogenerate -m "initial schema"`, open the
generated file and actually read it – autogenerate sometimes gets geometry/vector column
types wrong, so verify by eye before applying. Then `alembic upgrade head`.

Prompt 1.3 – Pydantic schemas

Build `app/schemas/listing.py`, `broker.py`, `customer.py`. For each: a Base schema with shared fields, a Create schema for input (Listing's Create schema takes `lat` and `lng` as separate floats, not the internal PostGIS type – the service layer converts them on write), and a Response schema for output (Listing's Response schema must include amenities, per AGENTS.md). Use `model_config = ConfigDict(from_attributes=True)` (or `orm_mode = True` if this repo ends up on Pydantic v1 – check what's actually installed) so these construct directly from ORM instances.

In `ListingCreate`, type `property_type` as `Literal["flat", "house_land"]`, not a plain string, so invalid values get rejected with a 422 before they ever reach the database. Full docstrings on every schema class.

Prompt 1.4 – Security and auth service

Build `app/core/security.py`:

- `hash_password(password: str) -> str` / `verify_password(plain: str, hashed: str) -> bool` via `passlib's CryptContext (bcrypt)`.
- `create_access_token(subject: str, expires_delta: timedelta | None = None) -> str`: build a JWT with `python-jose` embedding "sub" and "exp", signed with `settings.SECRET_KEY` / `settings.JWT_ALGORITHM`, defaulting expiry to `settings.JWT_EXPIRE_MINUTES`.
- `decode_access_token(token: str) -> dict`: decode and verify, raising a custom `InvalidTokenError` on any failure (expired, bad signature, malformed) instead of leaking `jose's` raw exception type.

Build `app/services/auth_service.py`:

- `register_user(db, email, password, role: Literal["broker", "customer"]) -> Broker | Customer`: check for an existing user with that email first (raise `DuplicateEmailError` if found), hash the password, create and commit the row, return it.
- `authenticate_user(db, email, password) -> Broker | Customer | None`: look up by email, verify password, return None on any failure – don't distinguish "no such user" from "wrong password" anywhere in the return value or eventual error message (security best practice against user enumeration; note this in a comment).

Build `app/api/deps.py`:

- `get_current_user(token = Depends(oauth2_scheme), db = Depends(get_db))`: decode via `decode_access_token`, look up the user, raise `HTTPException(401)` with a proper `WWW-Authenticate` header if invalid or the user no longer exists.

Prompt 1.5 – Auth routes

Build `app/schemas/auth.py` with a `RegisterRequest` (email, password, role) and rely on FastAPI's `OAuth2PasswordRequestForm` for login.

Build `app/api/routes/auth.py`:

POST `/auth/register`: call `auth_service.register_user()`, return an access token on success, 409 Conflict with a clear message if `DuplicateEmailError` is raised.

POST `/auth/login`: call `auth_service.authenticate_user()`, return 401 with a generic "incorrect email or password" message on failure, an access token on success.

Register this router in `app/main.py` with the exact prefixes from AGENTS.md: `/auth/register` and `/auth/login`.

Manual 1.2 – Manual auth test Use FastAPI's `/docs` Swagger UI to register a test broker and log in, confirming a token comes back.

Prompt 1.6 – Radius geo search

Build `app/services/geo_service.py`.

```
def search_by_radius(db: Session, lat: float, lng: float, radius_km: float, limit: int = 20) -> List[Broker]:
    """Search for brokers within a radius of a point.

    Validate lat in [-90, 90], lng in [-180, 180], radius_km positive and capped at a sane max (e.g. 100km – comment explaining this prevents an accidentally huge query). Query using GeoAlchemy2's ST_DWithin against a Point built from the input, casting both sides to ::geography so the distance argument (radius_km * 1000) is interpreted in meters rather than degrees – this is a common correctness bug, explain it clearly in a comment so nobody "simplifies" it incorrectly later. Order by distance ascending, limit to `limit`. Full docstring and type hints. Note in a comment that this will get a spatial index in Phase 5.
    """
```

Prompt 1.7 – Listing service

Build `app/services/listing_service.py`.

```
def create_listing(db: Session, broker_id: int, data: ListingCreate) -> Listing:
    """Create a new listing for a broker.

    Convert data.lat/data.lng into a WKTElement Point (srid=4326) via geoalchemy2, build the Listing ORM object with all fields from `data` plus broker_id, commit, refresh, return it. This function must NOT call Person"""
```

1's embedding/amenities pipelines itself – **return** the created listing **and** **let** the calling route trigger those **as** background tasks, so **this** service stays focused **on** persistence only. Document **this** separation clearly **in** the docstring.

```
def get_listing(db: Session, listing_id: int) -> Listing | None: simple
lookup, return None if missing – let the route decide how to turn that
into a 404.
```

```
def search_listings(db: Session, filters: dict, limit: int = 20) -> list[Listing]:
a general filtered search; implement price_min/price_max and property_type
filtering now, structured so more filters can be added later without a
rewrite.
```

Prompt 1.8 – Listings routes

Build app/api/routes/listings.py.

POST `/listings` (authenticated; current user must be a broker, **403** otherwise): accept `ListingCreate`, call `listing_service.create_listing()`, then queue two of Person 1's pipeline functions **as** FastAPI `BackgroundTasks`: `ai.embeddings.embed_listings.embed_single(listing.id)` – not `run()`, which is only **for the** initial batch pass over seeded data, not per-listing **use** – and `ai.amenities.amenities_service.fetch_and_cache(listing.id)`, which itself triggers a second re-embed of that listing once amenities finish **caching** (so the listing's search text ends up amenities-aware without this route needing to know that detail). Wrap the import itself **in** `try/except ImportError` so this endpoint still works during isolated development before Person 1's `ai/` package physically exists alongside this repo; log a clear warning when skipped. Return the created listing via `ListingResponse`, status **201**.

GET `/listings/{listing_id}`: call `listing_service.get_listing()`, **404** with a clear message **if** missing, otherwise `ListingResponse` (including amenities).

GET `/listings/search`: `lat`, `lng`, `radius_km` **as** validated query **parameters** (FastAPI's `Query()`), call `geo_service.search_by_radius()`, **return** a list of `ListingResponse`.

Register the router **in** `app/main.py`.

Prompt 1.9 – Synthetic data seeding

Build `app/db/seed_synthetic_data.py`.

`REAL_CITY_COORDS`: a hardcoded list of 20-30 real lat/lng points across actual neighborhoods in Surat, Gujarat – this is the one city the whole project uses for now (e.g. Adajan, Vesu, Citylight, Piplod, Athwa, Varachha – use real points within these areas, not the city center repeated). Person 1 is using the same city in their own local test seed, so keep this list to real Surat coordinates only, not a placeholder to swap for another city later.

```
def seed(n_listings=100, n_brokers=10, n_customers=30) -> None:
```

Using Faker: create brokers with realistic names/emails and a hashed dummy password; create customers similarly; create listings each assigned to a random broker, location from `REAL_CITY_COORDS` with a small random jitter (a few hundred meters, so listings aren't stacked on identical points), varied `property_type`, prices loosely correlated with `property_type/area` (not purely random – should look plausible), room/dimension data matching `property_type`. Leave embedding and amenities NULL – those are populated by Person 1's pipelines, not this script.

Before inserting, check for existing rows and require a `--force` CLI flag to proceed if the table isn't empty, so repeated runs don't keep duplicating data. Print a summary at the end. Add a `__main__` block with `argparse` for all the counts plus `--force`.

Manual 1.3 – Seed and spot-check Run the seed script, spot-check a few rows via `psql` or a DB GUI client.

Prompt 1.10 – Test suite

Build `tests/test_auth.py` and `tests/test_listings.py` using `pytest` and FastAPI's `TestClient`. Use a separate strategy for test isolation – either a second docker-compose Postgres service on a different port dedicated to tests, or wrapping each test in a transaction that's rolled back afterward – pick one, implement it, and explain the choice in a comment. Cover: register + login success; login failure on wrong password; creating a listing as an authenticated broker succeeds; creating a listing without auth returns 401; fetching a listing by id; fetching a nonexistent id returns 404; radius search returns listings within range and excludes ones outside it (seed 2-3 listings at known coordinates specifically for this test, don't rely on the full synthetic seed for precision).

Phase 2 — AI Integration (Weeks 5-6)

Prompt 2.1 — Chat endpoint

```
Build app/schemas/chat.py: ChatRequest (conversation_id: int | None,
message: str, polygon: list[list[float]] | None = None), ChatResponse
(reply: str, listings: list[ListingResponse]).
```

```
Build app/api/routes/chat.py.
```

```
POST /chat (authenticated as a customer):
```

1. Load the existing Conversation **by** conversation_id **if** given, **else** create a **new** one **for** the current user.
2. Append the incoming message **to** messages (role "user", content, timestamp), commit.
3. **If** polygon **is** provided, **call** `ai.rag.boundary_filter.get_candidates(polygon)` **for** candidate_ids – wrap the import **and** the **call in try/except** so a missing **or** failing `ai/` package doesn't break the endpoint; on failure, log a warning and fall back **to** candidate_ids=None.
4. **Call** `ai.rag.retriever.search(message, candidate_ids=candidate_ids)`.
5. **Call** `ai.inference.generate_response(message, listings, history=the conversation's prior messages)` for the reply – but first, add a `PLACEHOLDER_MODE` setting (env-var controlled, **default True**) that, **when** enabled, returns a simple canned response referencing the retrieved listings **by** title instead **of** calling the real model. This lets you build **and** fully test this endpoint before Person 1's fine-tuned model exists. Flip `PLACEHOLDER_MODE` **off** once it's available.
6. Append the assistant's reply to messages, commit.
7. **Return** `ChatResponse(reply=reply, listings=listings)`.

Wrap the AI-calling section **in try/except** so any AI-layer failure returns a clean **error** response **to** the frontend (**not** a raw stack trace), **while** logging full details server-side **for** debugging.

Manual 2.1 — Test in placeholder mode, then for real Test via `/docs` with

`PLACEHOLDER_MODE=true`. Once Person 1's `ai/` package is available for local integration testing (or after the eventual merge), set it to `false` and retest with the real model.

Phase 3 — Analytics & Boundary Search (Weeks 7-8)

Prompt 3.1 – Boundary search

Build `app/schemas/boundary.py`: `BoundarySearchRequest` with polygon: `list[list[float]]`, and a Pydantic field_validator enforcing: at least 3 points; each point a 2-element `[lat, lng]` with `lat` in `[-90,90]`, `lng` in `[-180,180]`; if `first != last` point, auto-close the ring by appending the `first` point rather than erroring (a common frontend omission).

Extend `app/services/geo_service.py`:

```
def search_by_boundary(db: Session, polygon: list[list[float]], limit: int = 50) -> list[Listing]:
    Build a WKT POLYGON string from the validated points, run
    ST_Contains(ST_GeomFromText(:wkt, 4326), location) via a parameterized
    query, limit results. Before querying, construct the polygon with Shapely
    and check .is_valid – reject self-intersecting shapes with a clear
    ValueError before ever touching the database.
```

Build `app/api/routes/boundary_search.py`:

```
POST /listings/search-boundary: accept BoundarySearchRequest, call
search_by_boundary(), catch ValueError from an invalid polygon and return
it as 422 with a clear message, otherwise return a list of
ListingResponse. Register at the exact path from AGENTS.md.
```

Manual 3.1 – Test boundary search Test with a plausible hand-drawn-style polygon around some seeded listings via `/docs`, confirm only listings inside come back. Also test a deliberately self-intersecting polygon to confirm the 422 path.

Prompt 3.2 – Broker analytics

Build `app/schemas/analytics.py`: a response schema with `total_listings`, `total_leads`, and a per-listing breakdown (`listing_id`, `title`, `lead_count`).

Build `app/services/analytics_service.py`:

```
def get_broker_analytics(db: Session, broker_id: int) -> dict: query all
listings for this broker, count associated Lead rows per listing via a
grouped aggregate query, return the structured dict described above.
```

Build `app/api/routes/analytics.py`:

```
GET /brokers/{broker_id}/analytics (authenticated; the current user must
match broker_id – 403 otherwise, comment noting this is a basic ownership
check, not full RBAC): call get_broker_analytics(), 404 if the broker
doesn't exist. Register the router.
```

Phase 4 — Amenities Data Layer (Week 9)

Prompt 4.1 — Confirm amenities schema and trigger

The amenities JSON column **on** Listing, **and** the background task **in** POST /listings that calls `ai.amenities.amenities_service.fetch_and_cache()` (**and**, inside that, a second background re-embed via `ai.embeddings.embed_listings.embed_single()`), were already **set** up back **in** Phase 1's listings route — this phase is just verification, since amenities weren't actually testable until Person 1's `amenities_service` exists.

Confirm the amenities JSON column exists **on** Listing — **if** the initial migration predates that field being added **to** the model, generate a **new** one **now**: ``alembic revision --autogenerate -m "add amenities column"`, review it, apply it.`

Verify `ListingResponse` (`app/schemas/listing.py`) includes the amenities field — it should **from** Phase 1, confirm rather than assume.

Add a code comment **on** the Listing model: **if** a future PATCH endpoint ever allows editing a listing's location, that path must call `fetch_and_cache(listing.id, force=True)` afterward so amenities data doesn't go stale. No such endpoint exists yet — this is a flag for whoever builds one later.

Manual 4.1 — End-to-end check Once Person 1's `amenities_service` is available for integration testing, create a listing with real-world coordinates via POST /listings, wait briefly, then GET /listings/{id} and confirm `amenities` is populated. If not yet testable in isolation, note this and revisit after the repos are merged.

Phase 5 — Performance, Testing & Hardening (Week 10)

Prompt 5.1 — Spatial index

Generate a new Alembic migration adding a GiST index on `Listing.location`: `op.create_index('ix_listing_location', 'listing', ['location'], postgresql_using='gist')`. Apply it. Then build `app/db/_explain_check.py` that runs EXPLAIN ANALYZE on a representative radius-search query and a representative boundary-search query against the seeded dataset, printing the plan so I can confirm it shows an Index Scan rather than a Seq Scan.

Prompt 5.2 – Load test

Build `app/db/_load_test.py`: fire 100 requests (sequential or via `concurrent.futures` – pick one and justify) at `GET /listings/search` and `POST /listings/search-boundary` against the running `local` server, and `print` min/median/p95/max latency `for` each. This doesn't need to be a polished framework – it needs real numbers I can put in the report and use to catch anything pathologically slow.

Manual 5.1 – Run and react Run the load test, review results. If anything's alarmingly slow, go back to the relevant service prompt with Antigravity and ask for a fix.

Phase 6 – Documentation & Submission Support (Weeks 11-13)

Prompt 6.1 – Architecture write-up

Write `backend/docs/backend-architecture.md` `for` a `final-year` academic report audience – clear, structured, technically precise. Cover: (1) the schema design `and` why (geometry/vector `column` choices, the `property_type` branching); (2) the `full` endpoint list `and` request/response shapes; (3) the auth approach; (4) the `boundary-search` design `and` polygon validation approach; (5) the amenities-caching `trigger` design `and` why it's background-task-based rather than synchronous; (6) the AI-layer bridge design (direct Python import vs. a network call, and why); (7) performance results from Phase 5's load test, `with` actual numbers; (8) known limitations. Pull `real` specifics `from` this repo's actual code, not generic descriptions.

Prompt 6.2 – Final cleanup pass

Review `every` file `in` `app/` `and` report: (1) `any` hardcoded secrets – should be zero; (2) leftover debug prints `or` commented-out dead code; (3) unresolved `TODO/FIXME` comments `with` file `and` line; (4) `any` endpoint whose actual path `or` response shape has drifted `from` `AGENTS.md`; (5) `any` function missing a docstring `or` type hints; (6) confirm `every` external call (DB queries, the AI-layer import) has error handling rather than an unhandled exception reaching the client. List `specific` findings, `not` a general assessment.

Manual 6.1 — Fix, review, ship Fix what's flagged. Review the write-up so it reflects your own understanding before your mentor reads it. Final `git add . && git commit -m "... " && git push .`. Coordinate the `git subtree merge` into the unified repo with the team.